Tutorial: rede convolucional com fotos de flores no Keras

Objetivo

Construir, treinar e avaliar uma rede convolucional (CNN) que reconhece cinco tipos de flores e testá-la com fotos da internet. O tutorial segue a receita do Keras em oito passos. Em cada passo há uma explicação, um código para executar e uma interpretação do resultado.

Como utilizar

- Abra um notebook novo no Google Colab e copie cada bloco de código deste documento para uma célula, na ordem em que aparecem. Execute a célula em seguida.
- Leia a explicação antes de executar o código e a interpretação depois.
- Ao final de cada passo, responda às questões indicadas no formulário do Google Forms (o link será informado pelo professor).
- O treino pode levar alguns minutos. Se estiver lento, o Colab permite ativar uma GPU no menu do ambiente de execução (opcional).
- A acurácia e o tempo de treino variam de uma execução e de um computador para outro. Por isso, as questões do formulário não pedem esses valores.

A receita do Keras

Receita	Neste tutorial
1. Carregar os dados	Passos 1, 2 e 3 (baixar, carregar e dividir em treino e teste)
2. Preparar os dados	Passo 4
3. Montar o modelo	Passo 5
4 e 5. Compilar e treinar	Passo 6
6. Avaliar	Passo 7
7. Prever	Passo 8 (com fotos da internet)

Os dados

O conjunto é o de fotos de flores do tutorial oficial de classificação de imagens do TensorFlow: 3670 fotos de cinco classes, organizadas em uma subpasta por classe.

Pasta	Flor
`daisy`	margarida
`dandelion`	dente-de-leão
`roses`	rosa
`sunflowers`	girassol
`tulips`	tulipa

Passo 1: baixar e conhecer os dados

A célula abaixo baixa o arquivo (cerca de 229 MB), extrai as fotos para a pasta `dados_flores` e conta quantas fotos há em cada classe. O nome de cada subpasta é o nome da classe.

CÓDIGO

```
import tarfile
import pathlib
import tensorflow as tf

url = "https://storage.googleapis.com/download.tensorflow.org/example_images/
flower_photos.tgz"
caminho = tf.keras.utils.get_file("flower_photos.tgz", origin=url, extract=False)

with tarfile.open(caminho) as t:
    t.extractall("dados_flores", filter="data")

pasta = pathlib.Path("dados_flores/flower_photos")
traducao = {"daisy": "margarida", "dandelion": "dente-de-leão", "roses": "rosa",
"sunflowers": "girassol", "tulips": "tulipa"}

total = 0
for classe in sorted(p for p in pasta.iterdir() if p.is_dir()):
    n = len(list(classe.glob("*.jpg")))
    total += n
    print(f"{classe.name:12s} {n:4d}  ({traducao[classe.name]})")
print("total de imagens:", total)
```

Interpretação. Os números esperados são `daisy 633`, `dandelion 898`, `roses 641`, `sunflowers 699` e `tulips 799`, com um total de 3670 imagens. As classes não têm exatamente o mesmo tamanho, mas nenhuma delas domina o conjunto.

A célula seguinte mostra uma foto de cada classe.

CÓDIGO

```
import matplotlib.pyplot as plt
from PIL import Image

fig, eixos = plt.subplots(1, 5, figsize=(15, 3.5))
for eixo, classe in zip(eixos, sorted(p for p in pasta.iterdir() if p.is_dir())):
    primeira_foto = sorted(classe.glob("*.jpg"))[0]
    eixo.imshow(Image.open(primeira_foto))
    eixo.set_title(f"{classe.name} ({traducao[classe.name]})")
    eixo.axis("off")
plt.tight_layout()
plt.show()
```

Interpretação. As fotos originais podem ter tamanhos e enquadramentos diferentes, o que será resolvido no passo seguinte.

> **Formulário.** Responda agora às questões 1 e 2 do formulário (Passo 1).

Passo 2: carregar as imagens como arrays

A rede exige que todas as imagens tenham o mesmo formato. A célula abaixo percorre as pastas e, para cada foto:

1. abre o arquivo e converte para RGB, isto é, 3 canais de cor (vermelho, verde e azul);
2. redimensiona para 96 × 96 pixels;
3. guarda a imagem em `x` e, em `y`, o número da classe (0 a 4, na ordem alfabética das pastas).

Pode levar alguns instantes.

CÓDIGO

```
import numpy as np
from PIL import Image

TAMANHO = (96, 96)

nomes_classes = sorted(p.name for p in pasta.iterdir() if p.is_dir())

imagens, rotulos = [], []
for indice, classe in enumerate(nomes_classes):
    for arquivo in sorted((pasta / classe).glob("*.jpg")):
        img = Image.open(arquivo).convert("RGB").resize(TAMANHO)
        imagens.append(np.array(img))
        rotulos.append(indice)

x = np.array(imagens)
y = np.array(rotulos)

print(x.shape, y.shape)
print(nomes_classes)
print(np.bincount(y))
```

Interpretação. `x.shape` deve ser `(3670, 96, 96, 3)`: 3670 imagens de 96 × 96 pixels com 3 canais. `y.shape` deve ser `(3670,)`, um número de classe por imagem. O último comando repete as contagens por classe: `[633 898 641 699 799]`.

Formulário. Responda agora às questões 3 e 4 do formulário (Passo 2).

Passo 3: dividir em treino e teste

Se a rede for avaliada com as mesmas fotos usadas para treiná-la, não saberemos se ela aprendeu ou apenas decorou. Por isso, 80% das fotos servem para o **treino** e 20% ficam guardadas para o **teste**, como uma prova com questões novas.

- `test_size=0.2` define que 20% das fotos vão para o teste.
- `random_state=42` fixa o sorteio, para que o resultado seja o mesmo a cada execução.
- `stratify=y` mantém a proporção de cada classe nos dois conjuntos.

CÓDIGO

```
from sklearn.model_selection import train_test_split

x_treino, x_teste, y_treino, y_teste = train_test_split(x, y, test_size=0.2,
random_state=42, stratify=y)

print(x_treino.shape, x_teste.shape)
print(y_treino.shape, y_teste.shape)
```

Interpretação. O treino deve ter formato `(2936, 96, 96, 3)` e o teste `(734, 96, 96, 3)`. Os rótulos têm formatos `(2936,)` e `(734,)`.

> **Formulário.** Responda agora às questões 5 e 6 do formulário (Passo 3).

Passo 4: preparar os dados

Os pixels vão de 0 a 255. A rede aprende melhor com valores entre 0 e 1, por isso cada valor é dividido por 255.0. A conversão para `float32` reduz o uso de memória: as imagens de treino ocupam cerca de 325 MB nesse formato. A mesma preparação será aplicada à foto testada no Passo 8.

CÓDIGO

```
x_treino_n = x_treino.astype("float32") / 255.0
x_teste_n = x_teste.astype("float32") / 255.0

print(x_treino_n.min(), x_treino_n.max())
```

Interpretação. A saída `0.0 1.0` confirma a nova faixa de valores.

> **Formulário.** Responda agora às questões 7 e 8 do formulário (Passo 4).

Passo 5: montar a rede convolucional

A rede é uma pilha de camadas:

Camada	O que faz
`Input(shape=(96, 96, 3))`	define o formato da foto colorida que entra
`Conv2D(32, 64, 128)`	filtros 3 × 3 que deslizam pela imagem procurando padrões; a rede aprende os filtros no treino
`MaxPooling2D((2, 2))`	reduz a altura e a largura pela metade, guardando o maior valor de cada bloco 2 × 2
`Flatten()`	estica o resultado em uma fila de números
`Dense(5, activation="softmax")`	5 saídas, uma por classe; o softmax as transforma em probabilidades que somam 1

CÓDIGO

```
from tensorflow import keras
from tensorflow.keras import layers

modelo = keras.Sequential([
    keras.Input(shape=(96, 96, 3)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(5, activation="softmax"),
])
```

A célula seguinte exibe o resumo do modelo: as camadas, o formato de saída de cada uma e o número de parâmetros.

CÓDIGO

```
modelo.summary()
```

Interpretação. A imagem encolhe a cada bloco e o número de filtros aumenta: a saída das camadas é `(94, 94, 32)`, `(47, 47, 32)`, `(45, 45, 64)`, `(22, 22, 64)`, `(20, 20, 128)` e `(10, 10, 128)`. O `Flatten` gera 12.800 valores, e a camada final os converte em 5 probabilidades. O total é de 157.253 parâmetros.

Formulário. Responda agora às questões 9 a 11 do formulário (Passo 5).

Passo 6: compilar e treinar

A compilação define como a rede aprende:

- `optimizer="adam"`: o método que ajusta os pesos da rede;
- `loss="sparse_categorical_crossentropy"`: a medida do erro que a rede tenta reduzir (apropriada para várias classes com rótulos inteiros);
- `metrics=["accuracy"]`: a acurácia, isto é, a porcentagem de acertos.

No treino, `epochs=10` faz a rede passar 10 vezes por todas as imagens de treino. Com `validation_data`, o Keras também mede o desempenho nas imagens de teste ao final de cada época. Elas são apenas medidas: a rede não aprende com elas.

CÓDIGO

```
modelo.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])

historico = modelo.fit(x_treino_n, y_treino, epochs=10, validation_data=(x_teste_n,
y_teste))
```

Interpretação. A cada época são exibidos quatro valores: `loss` e `accuracy` (nas imagens de treino) e `val_loss` e `val_accuracy` (nas imagens de teste). Espera-se que a perda de treino diminua e a acurácia de treino aumente ao longo das épocas. Observe também o que acontece com `val_accuracy`. Anote o tempo de cada época.

Formulário. Responda agora às questões 12 e 13 do formulário (Passo 6).

Passo 7: avaliar

O método `evaluate` mede o desempenho nas imagens de teste ao final do treino.

CÓDIGO

```
perda, acuracia = modelo.evaluate(x_teste_n, y_teste, verbose=0)
print(f"Acurácia no teste: {acuracia:.2%}")
```

A célula abaixo mostra, época a época, a acurácia e a perda no treino e no teste.

CÓDIGO

```
import matplotlib.pyplot as plt

epocas = range(1, len(historico.history["accuracy"]) + 1)

fig, eixos = plt.subplots(1, 2, figsize=(12, 4))
eixos[0].plot(epocas, historico.history["accuracy"], label="treino")
eixos[0].plot(epocas, historico.history["val_accuracy"], label="teste")
eixos[0].set_title("Acurácia por época")
eixos[0].set_xlabel("época")
eixos[0].legend()
eixos[1].plot(epocas, historico.history["loss"], label="treino")
eixos[1].plot(epocas, historico.history["val_loss"], label="teste")
eixos[1].set_title("Perda por época")
eixos[1].set_xlabel("época")
eixos[1].legend()
plt.tight_layout()
plt.show()
```

Como ler.

- Treino alto e teste alto: a rede aprendeu a diferenciar as flores.
- Treino alto e teste muito menor: a rede decorou as imagens de treino. Esse problema se chama **overfitting** (sobreajuste).
- Se a acurácia de treino continua subindo enquanto a de teste para de subir ou cai, a rede está começando a decorar.

Para saber se o resultado é bom, compare-o com o **piso**: o acerto de quem não aprendeu nada.

CÓDIGO

```
import numpy as np

contagens = np.bincount(y_teste)
print("Chutar entre 5 classes: 20%")
print(f"Responder sempre a classe mais frequente: {contagens.max() / len(y_teste):.1%}")
```

Interpretação. Com 5 classes de tamanhos parecidos, quem chuta acerta cerca de 1 em 5, ou 20%. Responder sempre a classe mais frequente também fica perto de um quarto. Uma rede útil precisa ficar bem acima desse piso.

> **Formulário.** Responda agora às questões 14 a 16 do formulário (Passo 7).

Passo 8: testar fotos da internet

Agora a rede será testada com uma foto escolhida por você.

Escolha da foto. Fotos mais parecidas com as do treino tendem a funcionar melhor: uma única flor ou um grupo da mesma flor, nítida, bem iluminada e ocupando boa parte da imagem. Evite desenhos, fotos escuras e fotos com várias espécies.

Preparo. A foto passa pelos mesmos passos das imagens de treino: RGB, 96 × 96 pixels e divisão por 255. Sem o mesmo formato, o Keras acusa erro; com outra escala (0 a 255), não há erro, mas a resposta tende a perder o sentido.

Resposta. A rede tem 5 saídas, por isso sempre responderá uma das 5 flores, mesmo que a foto seja de outra espécie. A **confiança** é a probabilidade que a rede atribui à sua resposta.

Passos para enviar a foto: baixe a imagem da internet para o computador (ou envie a foto do celular para si mesmo e baixe no computador) e execute a célula abaixo.

CÓDIGO

```
import io
import numpy as np
from PIL import Image, ImageOps
import matplotlib.pyplot as plt
from google.colab import files

nomes_pt = [traducao[c] for c in nomes_classes]

enviado = files.upload()          # abre a janela para escolher a foto
nome = list(enviado.keys())[0]
foto = ImageOps.exif_transpose(Image.open(io.BytesIO(enviado[nome]))).convert("RGB")

entrada = np.array(foto.resize(TAMANHO)).astype("float32") / 255.0
probabilidades = modelo.predict(entrada.reshape(1, 96, 96, 3), verbose=0)[0]
classe = int(np.argmax(probabilidades))

plt.imshow(foto)
plt.title(f"O modelo acha que é: {nomes_pt[classe]} ({probabilidades[classe]:.0%} de
confiança)")
plt.axis("off")
plt.show()
```

Interpretação. O título mostra a flor prevista e a confiança. A célula seguinte mostra as 5 probabilidades.

CÓDIGO

```
import matplotlib.pyplot as plt

plt.bar(nomes_pt, probabilidades)
plt.ylabel("probabilidade")
plt.title("As 5 saídas da rede para a foto")
plt.xticks(rotation=20)
plt.show()
```

Interpretação. Se duas barras forem altas e próximas, a rede ficou em dúvida entre as duas flores. Teste outras fotos executando de novo as duas últimas células.

Formulário. Responda agora às questões 17 a 19 do formulário (Passo 8).

Resumo

- A receita do Keras é a mesma em qualquer rede: carregar, preparar, montar, compilar, treinar, avaliar e prever.
- Imagens em arquivos precisam ter o mesmo tamanho e 3 canais; a classe vem do nome da pasta.
- A divisão em treino e teste permite medir se a rede generaliza.
- A acurácia no treino não basta: compare-a com a do teste e com o piso.
- Uma foto nova deve receber o mesmo preparo do treino, e a rede responde sempre uma de suas classes.